

Unidad 2

Modelos de Sistemas

Sistemas Distribuidos

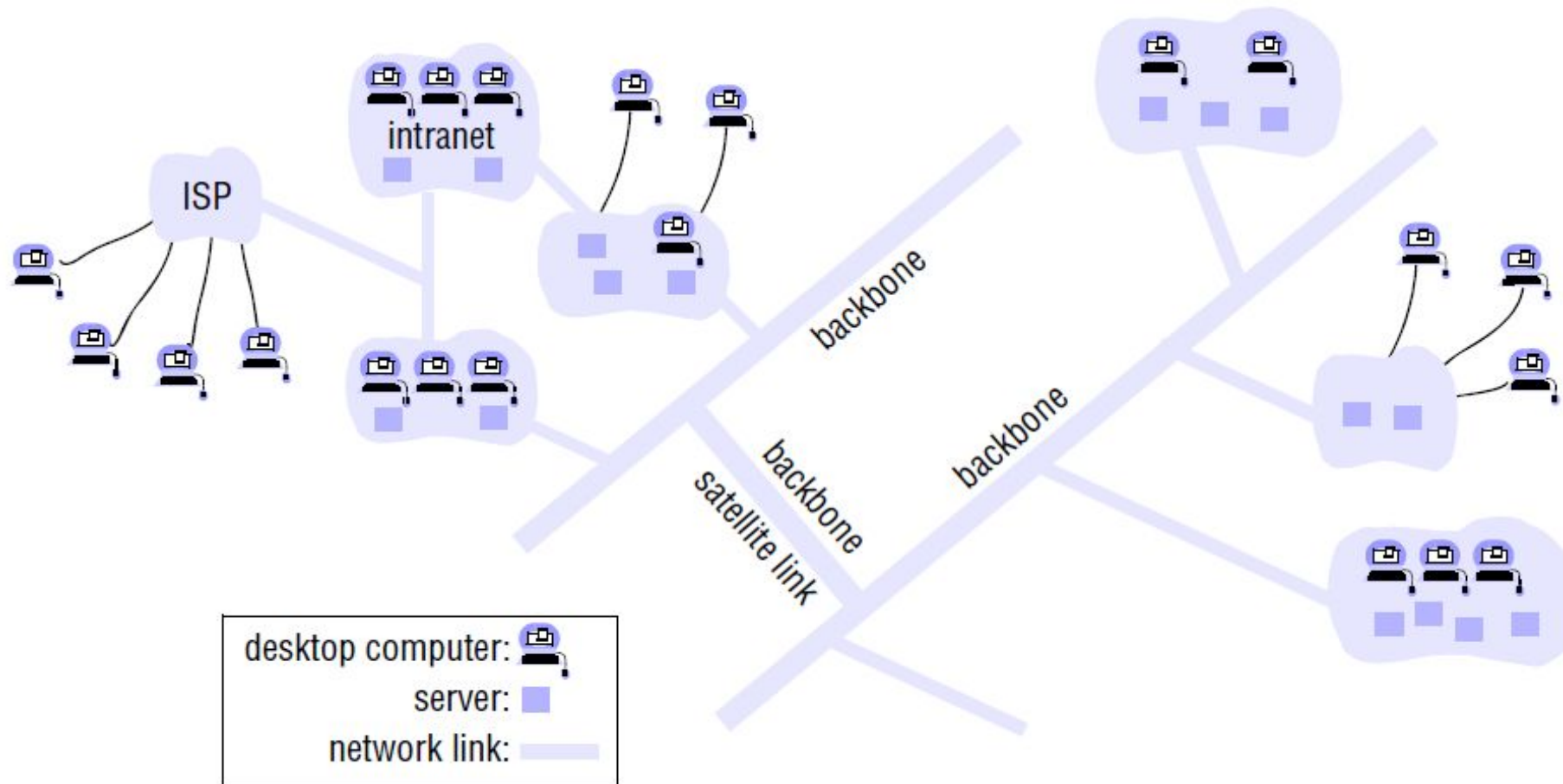
Universidad Nacional de Asunción, Facultad Politécnica

Ingeniería Informática

Ing. Fernando Mancía, Ing. Juan Ferreira

Introducción

Figure 1.3 A typical portion of the Internet



Introducción

- **Modelos Físicos:** es la forma más explícita de describir un sistema, incluye la composición de hardware de un sistema en términos de los ordenadores (y otros dispositivos, como teléfonos móviles) y sus redes de interconexión.
- **Modelos Arquitectónicos:** describen un sistema en términos de las tareas de cálculo y de comunicación realizadas por sus elementos computacionales. Los elementos pueden ser ordenadores individuales o conjuntos de ellos con sus interconexiones.
- **Modelos Fundamentales:** perspectivas abstractas con el fin de examinar los aspectos individuales de un SD.

Introducción

Estas cuestiones se consideran en tres modelos:

- **El modelo de interacción** trata de las prestaciones y de la dificultad de poner límites temporales en un sistema distribuido, por ejemplo para la entrega de mensajes.
- **El modelo de fallos** intenta dar una especificación precisa de los fallos que se pueden producir en los procesos y en los canales de comunicación. Define comunicación fiable y procesos correctos.
- **El modelo de seguridad** discute sobre las posibles amenazas para los procesos y los canales de comunicación. Introduce el concepto de canal seguro, que lo es frente a dichas amenazas.

Modelos Físicos

- **Un Modelo Físico** es una representación de los elementos de hardware subyacentes de un sistema distribuido que abstrae los detalles específicos de las tecnologías informáticas y las redes empleadas.
- Resulta útil identificar en tres generaciones de sistemas distribuidos.
 - **Primeros Sistemas Distribuidos:** emergieron entre los 70' y 80' en respuesta a la emergente tecnología LAN, específicamente Ethernet. Típicamente entre 10 a 100 nodos conectados por medio de una LAN.
 - **Sistemas Distribuidos en la escala de Internet:** surgieron en la década de los 90' en respuesta al impresionante crecimiento de Internet. Consiste en un conjunto de nodos interconectado por una red de redes. El nivel de Heterogeneidad comprende Redes, Arquitectura de Computadores, S.O.
 - **Sistemas Distribuidos Contemporáneos:** aparición de la computación móvil en la cual los nodos comprenden computadores personales, asistentes digitales, teléfonos móviles. La aparición de la computación en Nube, computación ubicua y tecnologías variadas de red, así como infinidad de aplicaciones hacen que la cantidad se expanda increíblemente.

Modelos Físicos

Figure 2.1 Generations of distributed systems

<i>Distributed systems:</i>	<i>Early</i>	<i>Internet-scale</i>	<i>Contemporary</i>
<i>Scale</i>	Small	Large	Ultra-large
<i>Heterogeneity</i>	Limited (typically relatively homogenous configurations)	Significant in terms of platforms, languages and middleware	Added dimensions introduced including radically different styles of architecture
<i>Openness</i>	Not a priority	Significant priority with range of standards introduced	Major research challenge with existing standards not yet able to embrace complex systems
<i>Quality of service</i>	In its infancy	Significant priority with range of services introduced	Major research challenge with existing services not yet able to embrace complex systems

Modelos Arquitectónicos

- La arquitectura de un sistema es su estructura en términos de componentes especificados por separado.
- El objetivo global es asegurar que la estructura satisfará las demandas presentes y previsibles sobre él.
- Un modelo arquitectónico de un sistema distribuido **simplifica y abstrae**, inicialmente, las funciones de los componentes individuales de dicho sistema y posteriormente considera:
 - La ubicación de los componentes en una red.
 - Las interrelaciones entre los componentes.

Modelos Arquitectónicos

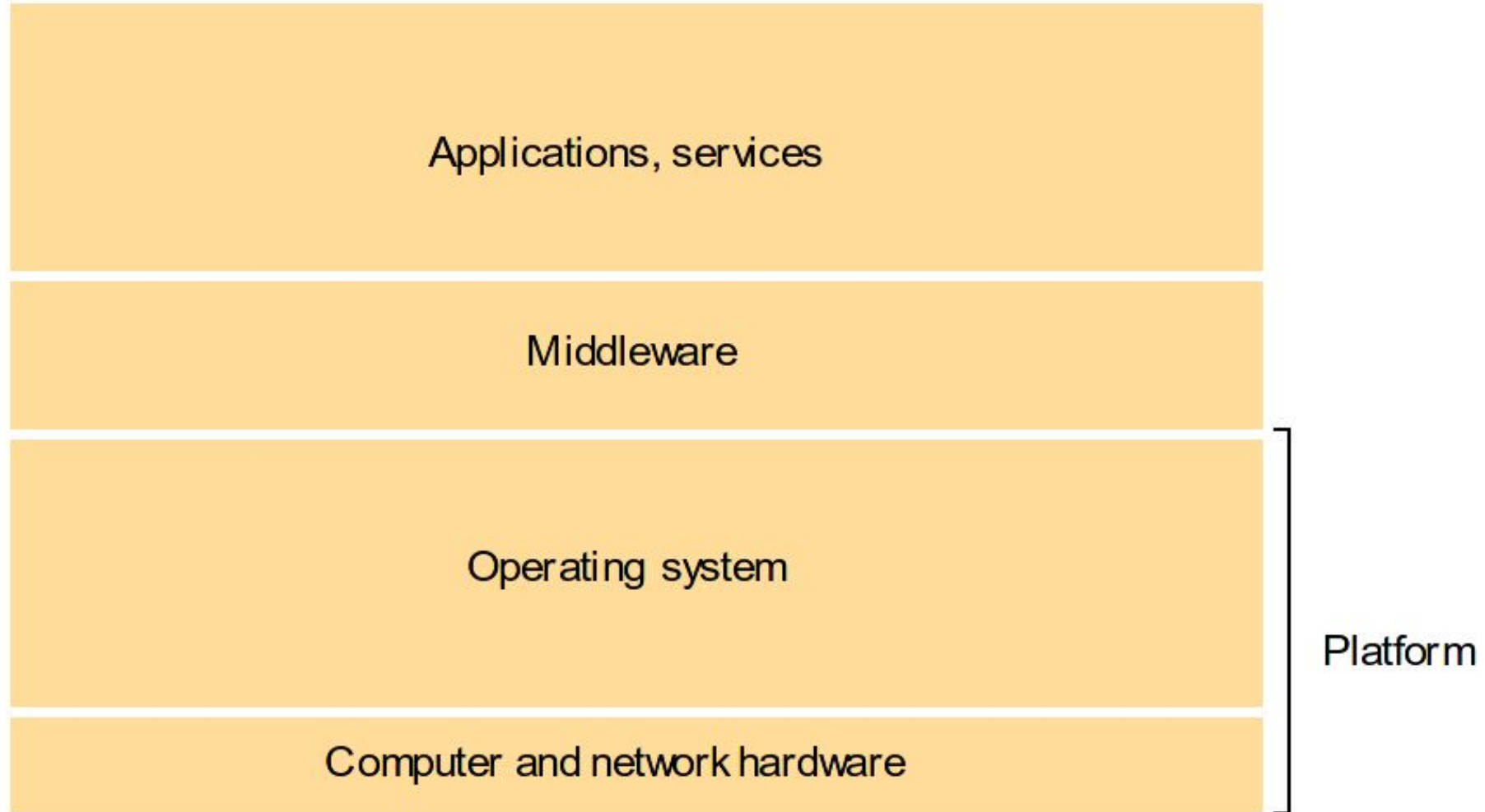
Arquitectura de Software



- El término “**arquitectura de software**” se refería inicialmente a la estructuración del software como capas o módulos en un único computador y más recientemente en términos de los servicios ofrecidos y solicitados entre procesos localizados en el mismo o diferentes computadores.
- Esta vista orientada a proceso y a servicio puede expresarse en términos de capas de servicio.
- Un servidor es un proceso que acepta peticiones de otros procesos.
- Un servicio distribuido puede proveerse desde uno o más procesos servidor, interactuando entre cada uno de ellos, y con los procesos clientes, para mantener una visión de los recursos del servicio consistente con el sistema.

Modelos Arquitectónicos

Patrones de Arquitectura



Modelos Arquitectónicos

Patrones de Arquitectura



- **Plataforma.** Al nivel de hardware y las capas más bajas de software se denominan “plataforma” para sistemas distribuidos y aplicaciones. Estas capas más bajas proporcionan servicios a las que están por encima de ellas, y que son implementadas independientemente en cada computador, proporcionando una interfaz de programación del sistema a un nivel que facilita la comunicación y coordinación entre procesos.
- **Middleware.** Es una capa de software cuyo propósito es enmascarar la heterogeneidad y proporcionar un modelo de programación conveniente para los programadores de aplicaciones. Se representa mediante procesos u objetos en un conjunto de computadores que interactúan entre sí para implementar mecanismos de comunicación y de recursos compartidos para aplicaciones distribuidas.

Modelos Arquitectónicos

Patrones de Arquitectura



- El middleware se ocupa de proporcionar bloques útiles para la construcción de componentes software que puedan trabajar con otros en un sistema distribuido.
- Mejora el nivel de las actividades de comunicación de los programas de aplicación soportando abstracciones como: procedimiento de invocación remota, comunicación entre un grupo de procesos, notificación de eventos, replicación de datos compartidos y transmisión de datos multimedia en tiempo real.

Modelo Arquitectónico

Roles y Responsabilidades

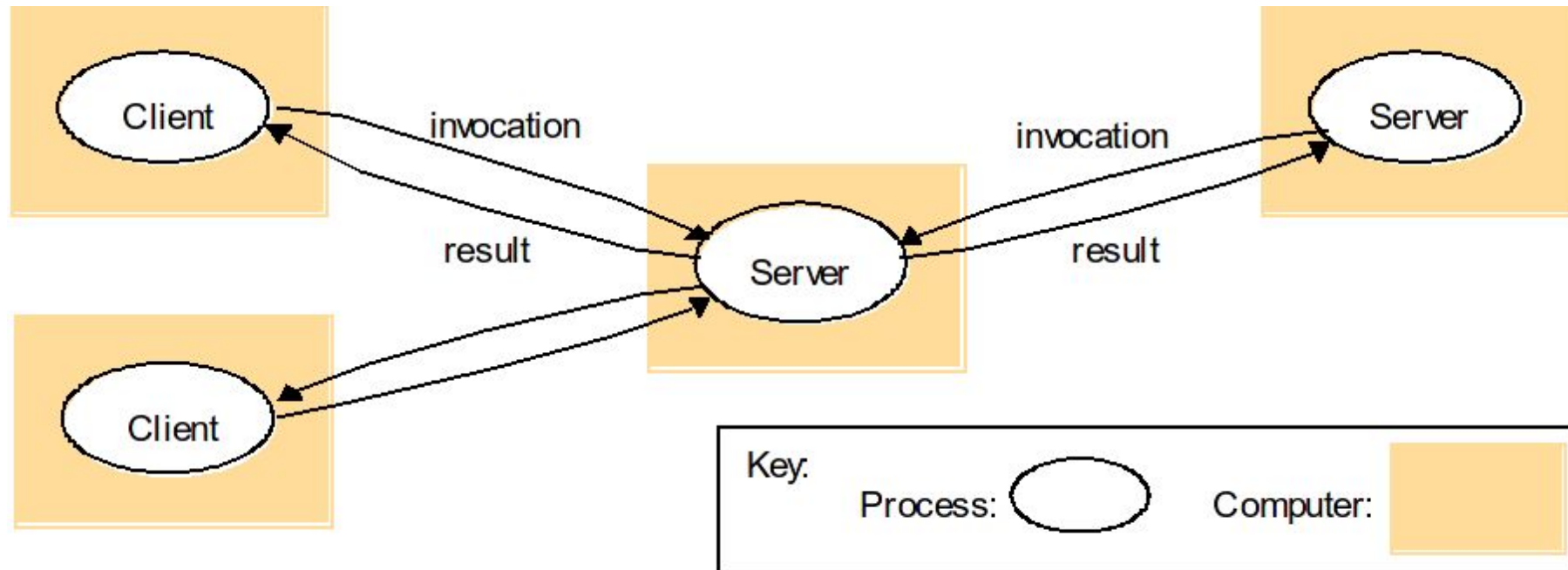


- El aspecto importante en el diseño de sistemas distribuidos es la división de responsabilidades entre los componentes del sistema.
- Consideraciones en la arquitectura:
 - La partición de datos o la replicación en servidores cooperativos.
 - El uso de caché para los datos en clientes y servidores proxy.
 - El uso de código y agentes móviles.
 - Los requisitos para añadir o eliminar dispositivos móviles de forma conveniente.
- No hay un tiempo global en un sistema distribuido.
- Los relojes en los computadores no tienen necesariamente el mismo tiempo.
- Toda comunicación entre procesos se realiza por medio de mensajes. La comunicación en una red puede verse afectada por: retrasos, puede sufrir variedad de fallos y es vulnerable a los ataques a la seguridad.

Modelo Arquitectónico

Roles y Responsabilidades

- Clientes que invocan a servidores individuales



Modelo Arquitectónico

Roles y Responsabilidades



Modelo cliente-servidor

- Históricamente la arquitectura cliente- servidor es la más importante, y continúa siendo la más ampliamente utilizada.
- La Figura 2.2 muestra la sencilla estructura sobre la que interaccionan los procesos clientes con los procesos servidores individuales, en computadores separados, con el fin de acceder a los recursos compartidos que ellos gestionan.
- Los servidores pueden, a su vez, ser clientes de otros servidores, como se indica en la figura. Por ejemplo, un servidor web es, a veces, un cliente de un servidor de archivos local que gestiona los archivos en los que están almacenadas las páginas web.
- Los servidores web y la mayoría del resto de los servicios de Internet son clientes del servicio DNS, que traduce Nombres de Dominio de Internet en direcciones de red.

Modelo Arquitectónico

Roles y Responsabilidades

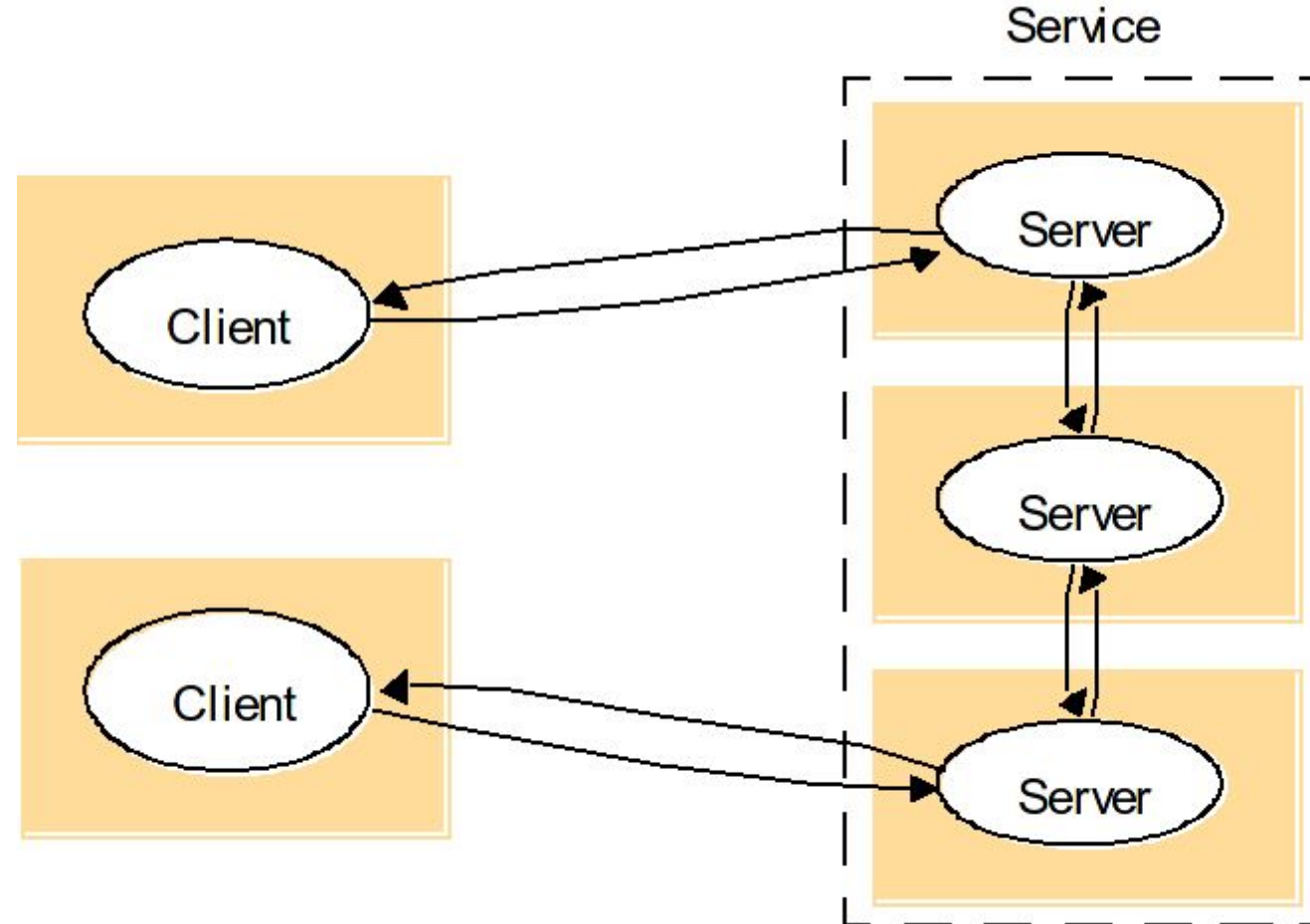


- Otro ejemplo relacionado con web son los buscadores, que permiten a los usuarios buscar resúmenes de la información disponible en las páginas web de los sitios. Estos resúmenes están realizados por programas llamados escaladores (crawlers) web, que se ejecutan en segundo plano en el sitio del buscador utilizando peticiones HTTP para acceder a los servidores web a través de Internet.
- Por tanto, una máquina de búsqueda es tanto un cliente como un servidor: responde a las consultas del navegador de los clientes y ejecuta web crawlers que actúan como clientes de otros servidores web.
- En este ejemplo, las tareas del servidor (responder a las consultas de usuarios) y las tareas de crawler (hacer peticiones a otros servidores web) son totalmente independientes; hay muy poca necesidad de sincronizarlas y pueden ejecutarse concurrentemente. De hecho, un buscador típico debiera incluir muchos hilos (threads) de ejecución concurrente, algunos sirviendo a sus clientes y otros ejecutando crawlers web.

Modelo Arquitectónico

Roles y Responsabilidades

- Un servicio proporcionado por múltiples servidores



Modelo Arquitectónico

Roles y Responsabilidades



Servicios proporcionados por múltiples servidores

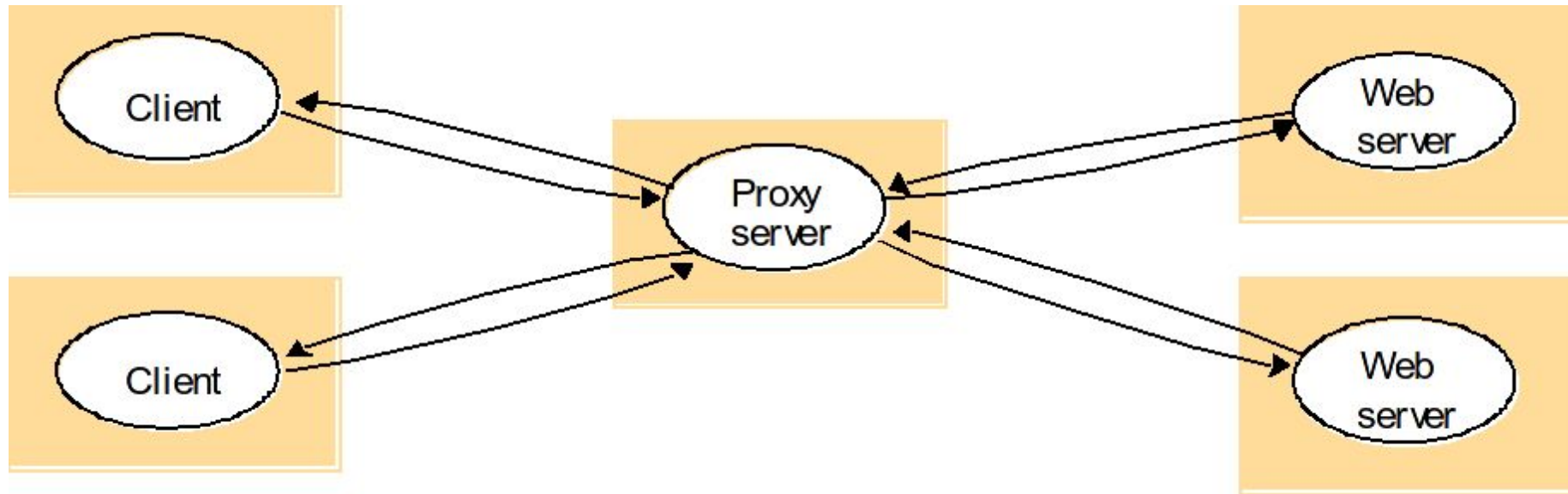
- Los servicios pueden implementarse como distintos procesos de servidor en computadores separados interaccionando, cuando es necesario, para proporcionar un servicio a los procesos clientes
- Los servidores pueden dividir el conjunto de objetos en los que está basado el servicio y distribuírselos entre ellos mismos, o pueden mantener copias replicadas de ellos en varias máquinas.
- **Partición:** La Web, ejemplo típico de **partición de datos**. Cada servidor web administra su propio conjunto de recursos. Un usuario puede emplear un navegador para acceder al recurso que está en cualquiera de los servidores.
- **Replicación:** se utiliza para aumentar las prestaciones y disponibilidad y para mejorar la tolerancia a fallos. Proporciona múltiples copias consistentes de datos en proceso que se ejecutan en diferentes computadores. Por ejemplo, el servicio web proporcionado por altavista.digital.com se encuentra redirigido hacia varios servidores con la base de datos replicada en memoria.

Modelo Arquitectónico

Roles y Responsabilidades



- Servidores proxy y cachés



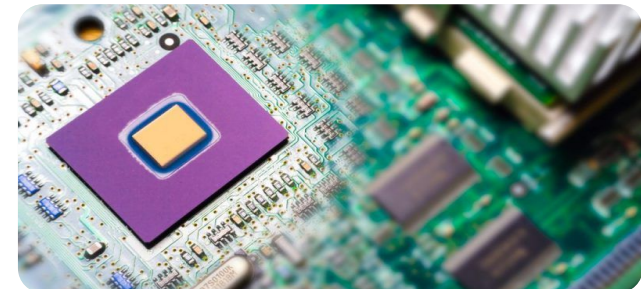
Modelo Arquitectónico

Roles y Responsabilidades



Servidores proxy y cachés

- Un caché es un almacén de objetos de datos utilizados recientemente, y que se encuentra más próximo que los objetos en sí. Al recibir un objeto nuevo en un computador se añade al almacén del caché, reemplazando, si fuera necesario, algunos objetos existentes.
- Cuando se necesita un objeto en un proceso cliente, el servicio caché comprueba inicialmente la caché y le proporciona el objeto de una copia actualizada. Si no, se buscará una copia actualizada. Los cachés pueden estar ubicados en cada cliente o en un servidor proxy que puede compartirse desde varios clientes.



Modelo Arquitectónico

Roles y Responsabilidades

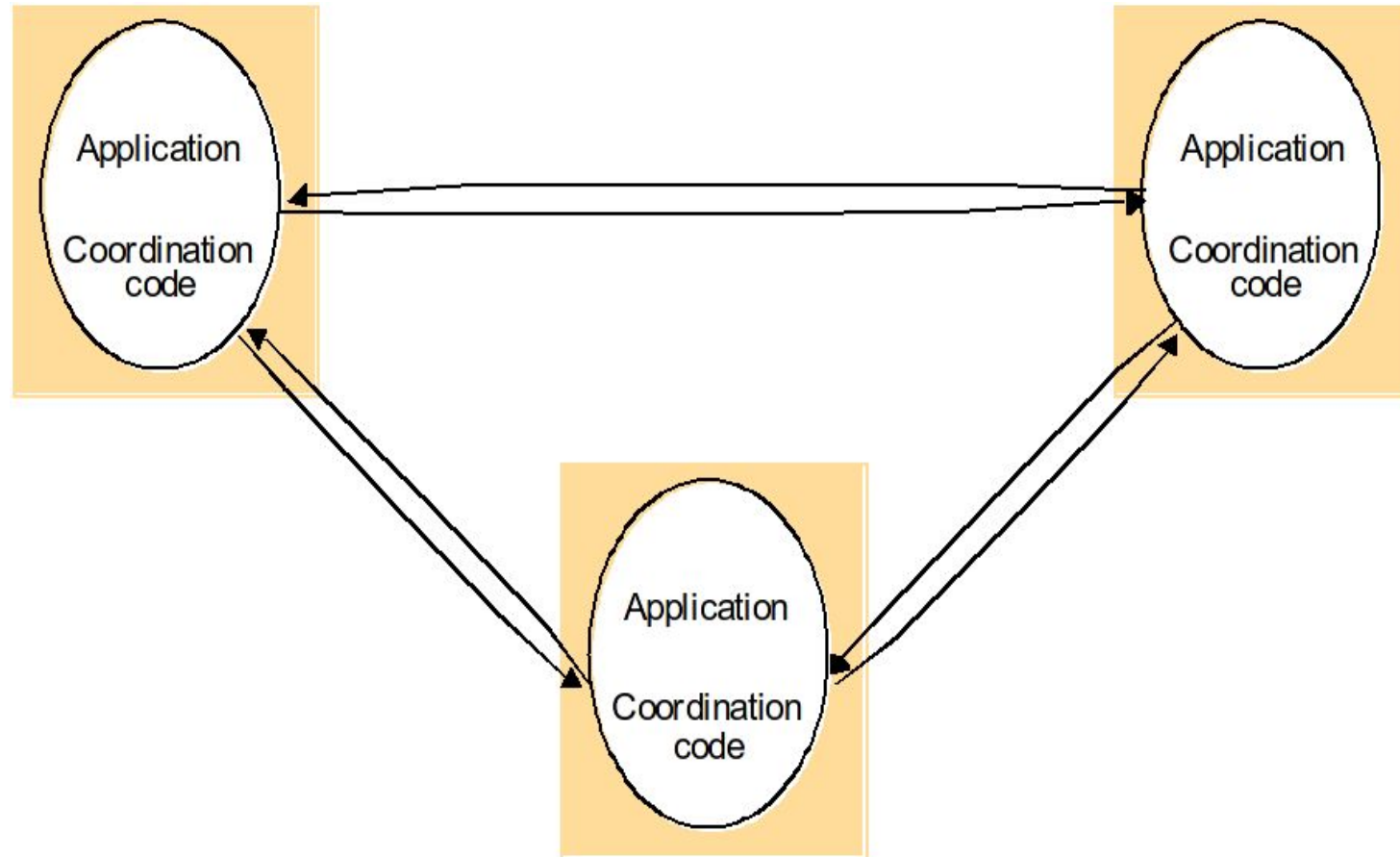


- Los servidores proxy para el web proporcionan un caché compartido de recursos web a las máquinas cliente de uno o más sitios.
- El propósito de los servidores proxy es incrementar la **disponibilidad y prestaciones del servicio**, reduciendo la carga en redes de área amplia y en servidores web.
- Los servidores proxy pueden desempeñar otros roles, por ejemplo, pueden ser utilizados para acceder a servidores web remotos a través de un cortafuegos.

Modelo Arquitectónico

Roles y Responsabilidades

- Procesos de igual a igual. Peer to Peer



Modelo Arquitectónico

Roles y Responsabilidades



Procesos de igual a igual. Peer to Peer

- En esta arquitectura todos los procesos desempeñan tareas semejantes, interactuando cooperativamente como iguales para realizar una actividad distribuida o cómputo sin distinción entre clientes y servidores.
- En este modelo, el código en los procesos parejos o «iguales» mantiene la consistencia de los recursos y sincroniza las acciones a nivel de la aplicación cuando es necesario.
- La Figura muestra un ejemplo con tres instancias de esta situación; en general, n procesos parejos podrán interactuar entre ellos, dependiendo el patrón de comunicación de los requisitos de la aplicación.
- La eliminación del proceso servidor reduce los retardos de comunicación entre procesos al acceder a objetos locales.

Modelo Arquitectónico

Roles y Responsabilidades



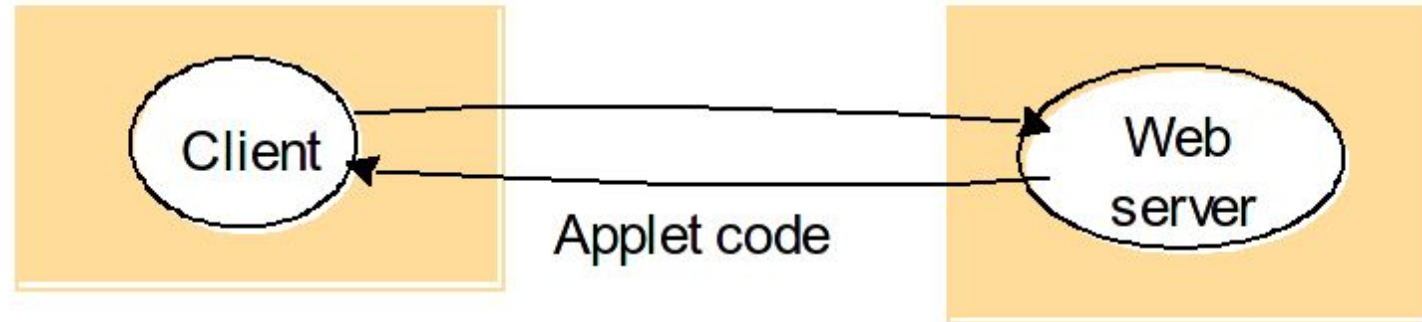
- Una aplicación de pizarra distribuida permite que usuarios en varios computadores vean y modifiquen interactivamente un dibujo que se comparte entre ellos.
- Esto puede implementarse con un proceso de aplicación en cada sitio y que se basa en capas de middleware para realizar **notificación de eventos y comunicación a grupos** para indicar a todos los procesos de la aplicación de los cambios en el dibujo.
- Esto proporciona, a los usuarios de objetos distribuidos compartidos, una respuesta interactiva mejor de la que se puede obtener con una arquitectura basada en servidor.

Modelo Arquitectónico

Roles y Responsabilidades

- Variaciones en el modelo cliente-servidor

a) client request results in the downloading of applet code



b) client interacts with the applet



Modelo Arquitectónico

Roles y Responsabilidades



Variaciones en el modelo cliente-servidor

- Podemos distinguir distintas variaciones del modelo cliente-servidor, dependiendo de la consideración de los factores siguientes:
 - El uso de código móvil y agentes móviles.
 - Las necesidades de los usuarios de computadores de coste bajo y con recursos hardware limitados, que son muy sencillos de manejar.
 - El requisito de añadir o eliminar de una forma conveniente dispositivos móviles.

Modelo Arquitectónico

Roles y Responsabilidades



Variaciones en el modelo cliente-servidor. Código móvil.

- Los applets son el ejemplo más conocido y más ampliamente extendido de código móvil; un usuario que ejecute un navegador y que seleccione un enlace con un applet, cuyo código esté almacenado en un servidor web, descargará el código en el navegador y se ejecutará allí .
- Una ventaja de ejecutar el código descargado localmente es que puede proporcionar una buena respuesta interactiva puesto que no sufre ni de los retardos ni de la variabilidad del ancho de banda asociados con la comunicación en red.
- Acceder a los servicios significa ejecutar código que pueda invocar sus operaciones.
- Consideremos una aplicación donde se necesite que los usuarios dispongan de información actualizada según se hagan cambios en el origen de los datos en el servidor. Esto no se puede conseguir mediante las interacciones normales con el servidor web, que siempre se inician por parte del cliente.

Modelo Arquitectónico

Roles y Responsabilidades



- La solución es utilizar software adicional que trabaja de una forma a la que a veces se refiere como modelo push, donde el servidor, en lugar del cliente, es el que inicia las interacciones.
- Por ejemplo, un corredor de bolsa puede proporcionar un servicio personalizado para notificar a los usuarios los cambios en las cotizaciones; para utilizar dicho servicio, cada usuario debiera haber descargado un applet especial, que recibe las actualizaciones del servidor del corredor, las presenta en pantalla al usuario y quizá realiza operaciones automáticas de compra y venta, desencadenadas por las condiciones prefijadas por el usuario y almacenados localmente en el computador del mismo.

Modelo Arquitectónico

Roles y Responsabilidades



Variaciones en el modelo cliente-servidor. Agentes móviles.

- Agente Móvil: es un programa en ejecución (lo que incluye tanto código como datos) que se traslada de un computador a otro en la red realizando una tarea para alguien; por ejemplo, recolectando información, y retornando eventualmente con los resultados.
- Un agente móvil puede hacer muchas solicitudes a los recursos locales de los sitios que visita, por ejemplo, accediendo a anotaciones individuales en una base de datos. Si se compara esta arquitectura con un cliente estático que realiza solicitudes de algunos recursos, transfiriendo posiblemente grandes cantidades de datos, hay una reducción en el coste de la comunicación y en el tiempo con la sustitución de las solicitudes remotas por las locales.

Modelo Arquitectónico

Roles y Responsabilidades



- Los agentes móviles (como el código móvil) son una amenaza potencial de seguridad para los recursos de los computadores que visitan.
- El entorno debe decidir que recursos locales estará permitido el acceso.
- La identidad de éste debe incluirse de una forma segura en el código y los datos del agente móvil.
- Los agentes móviles pueden ser vulnerables, y pueden no ser capaces de finalizar su tarea si se les niega el acceso a la información que necesitan.
- Las tareas realizadas por agentes móviles pueden realizarse por otros medios. Por ejemplo, los escaladores (crawlers) web que necesitan acceder a recursos en servidores web a través de Internet trabajan bastante bien realizando invocaciones remotas a los procesos del servidor.
- Por estas razones, la aplicabilidad de agentes móviles puede ser limitada.

Modelo Arquitectónico

Roles y Responsabilidades



- Un ejemplo de Agente Móvil, es el programa gusano worm desarrollado por XEROX PARC con el fin de utilizar computadores desocupados para realizar cálculos intensivos.
- Se pueden utilizar agentes móviles para instalar y mantener software en los computadores de una organización, o comparar los precios de los productos de un número de vendedores visitando el sitio de cada vendedor y realizando una serie de operaciones sobre la base de datos.

Modelo Arquitectónico

Roles y Responsabilidades

- Clientes ligeros

Network computer or
PC



network

Compute
server

Application
Processes



Modelo Arquitectónico

Roles y Responsabilidades



Variaciones en el modelo cliente-servidor. Clientes ligeros

- El término cliente ligero se refiere a una capa de aplicación que soporta una interfaz de usuario basada en ventanas sobre un computador local del usuario mientras se ejecutan programas de aplicación en un computador remoto.
- Las aplicaciones se ejecutan en un servidor de cómputo, un potente computador que tiene capacidad para ejecutar gran número de aplicaciones simultáneamente. El servidor de cómputo (multiprocesador o un sistema de computadores fuertemente acoplado (cluster)), ejecuta una versión multiprocesador de un sistema operativo como UNIX o Windows NT.
- El principal inconveniente de una arquitectura de cliente ligero se encuentra en actividades gráficas fuertemente interactivas, como CAD o proceso de imagen, en las que los retrasos experimentados por los usuarios se ven aumentados por la necesidad de transferir imágenes e información vectorial entre el cliente ligero y los procesos de aplicación, padeciendo latencias tanto de red como de sistema operativo.

Modelo Arquitectónico

Roles y Responsabilidades

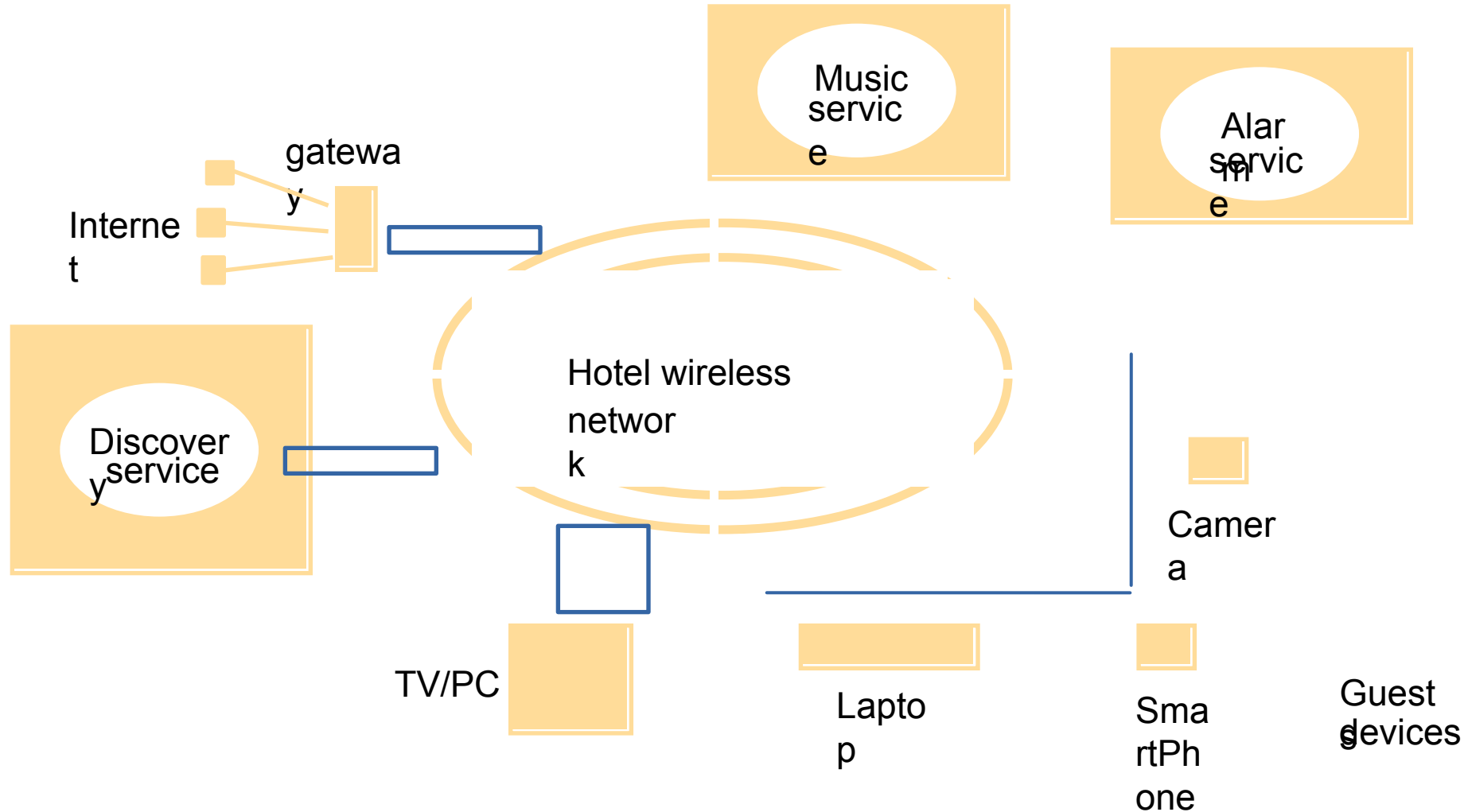


- **Implementaciones de clientes ligeros:** Los sistemas de clientes ligeros son sencillos en concepto y sus implementaciones son inmediatas en algunos entornos. Por ejemplo, la mayoría de las variantes de UNIX incluyen el sistema de ventanas X-11 .
- WinFrame de Citrix [www.citrix.com] es una implementación comercial del concepto de cliente ligero. Puede ejecutarse en una gran variedad de plataformas y soporta un entorno (desktop) que proporciona acceso interactivo a las aplicaciones corriendo en máquinas Windows NT.
- Otras aproximaciones incluyen los sistemas de computadores de red virtual (Teleporting y Virtual Network Computer, VNC) desarrollados en los laboratorios AT&T de Cambridge, Inglaterra [Richardson y otros, 1998]. VNC establece la frontera en el nivel de operaciones de píxeles en pantalla y mantiene el contexto gráfico para los usuarios cuando éstos se mueven entre computadores.

Modelo Arquitectónico

Roles y Responsabilidades

- Dispositivos móviles y enlace espontáneo



Modelo Arquitectónico

Roles y Responsabilidades



Dispositivos móviles y enlace espontáneo

- La figura muestra los dispositivos que el usuario ha llevado a la suite del hotel: un computador portátil, una cámara digital y una agenda portátil (PDA), cuyas posibilidades a través de interfaces inalámbricas le permiten actuar como un controlador universal.
- Las características esenciales de la conexión a red espontánea son:
- Conexión fácil a la red local: los enlaces sin cable eliminan la necesidad de cableado preinstalado y eliminan el inconveniente y las cuestiones de fiabilidad que rodean a los conectores y enchufes. Sólo existen aquellos enlaces que pueden mantenerse aunque haya movimiento. Un dispositivo colocado en un nuevo entorno de red se reconfigura de modo transparente para conectarse allí; el usuario no tiene que introducir los nombres o direcciones de los servicios locales para obtenerlo.
- Integración fácil con servicios locales: los dispositivos que se encuentran insertados (y moviéndose) entre redes existentes de dispositivos descubren automáticamente qué servicios se proporcionan, sin acciones especiales de configuración por el usuario.

Modelo Arquitectónico

Roles y Responsabilidades



- Para usuarios móviles se presentan otras cuestiones.
- **Conectividad limitada:** los usuarios no siempre están conectados cuando se desplazan. Por ejemplo, al viajar en un tren por un túnel pueden estar desconectados eventualmente de su red inalámbrica. También pueden desconectarse durante largos períodos de tiempo en zonas sin cobertura o cuando el permanecer conectado sea demasiado costoso.
- **Seguridad y privacidad:** en un escenario como el del huésped de un hotel se presentan muchas cuestiones de seguridad y privacidad personal. El hotel y sus huéspedes son vulnerables a los ataques de seguridad de los empleados y de otros huéspedes del hotel, que intenten conexiones inalámbricas de formas no supervisadas.

Requisitos de diseño para arquitecturas distribuidas

- a. **Prestaciones.** Los retos que surgen de la distribución de recursos aumentan la necesidad de gestión de actualizaciones concurrentes. Los temas de prestaciones se consideran bajo:
1. **Capacidad de respuesta (Responsiveness):** Los usuarios de aplicaciones interactivas necesitan rapidez y consistencia en las interfaces, pero, a menudo, los programas cliente necesitan acceder a recursos compartidos.
 2. **Productividad (Throughput):** Una medida tradicional de prestaciones para computadores es la productividad, la rapidez a la que se realiza el trabajo computacional.
 3. **Balance de cargas computacionales:** uno de los propósitos de los sistemas distribuidos es permitir que las aplicaciones y los procesos de servicio evolucionen concurrentemente sin competir por los mismos recursos y explotando los recursos computacionales disponibles (procesador, memoria y capacidades de red).

Requisitos de diseño para arquitecturas distribuidas

- b. **Calidad de servicio:** las principales propiedades no funcionales de los sistemas, que afectan a la calidad del servicio experimentado por los clientes y usuarios, son: fiabilidad, seguridad y prestaciones.
- c. **Aspecto de fiabilidad:**
 - b. **Tolerancia frente a fallos:** Las aplicaciones estables deben seguir funcionando aun en presente de fallos de hardware, software y redes.
 - c. **Seguridad:** Ubicar y proteger datos/recursos sensibles contra ataques.

Ejercicios de retroalimentación

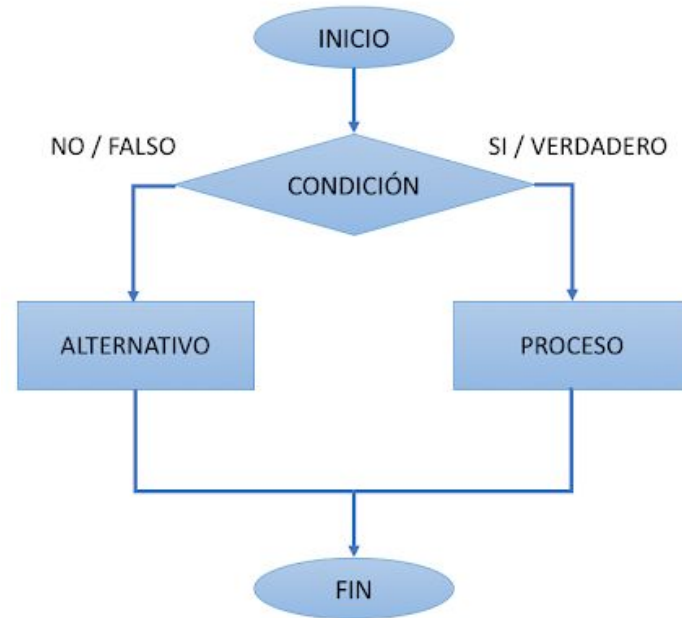
1. Construya un modelo distribuido Cliente-Servidor de una aplicación en internet (Google Mail)
2. Describa las responsabilidades de cada componente y cómo cooperan entre sí para proveer el servicio.
3. Pensando en Google Mail... ¿cuál sería la estrategia de particionamiento y/o replicación (uso de cache) en sus servidores?
4. Mencione los tipos de recursos locales que pueden ser vulnerables a un ataque de un agente móvil descargado de un sitio web remoto no confiable.
5. Si tuvieras que diseñar un video juego distribuido. Qué modelo arquitectónico elegirías? Mencione un par de ventajas y desventajas del modelo seleccionado.
6. En un sistema distribuido no existe un reloj global. Entonces, cuál podría ser una alternativa para sincronizar todos los componentes del sistema distribuido y aproximarnos a la existencia de un reloj global?

Modelos Fundamentales

- **Interacción:** el cómputo ocurre en los procesos; los procesos interaccionan por paso de mensajes, lo que resulta en comunicación y coordinación (sincronización y ordenamiento de actividades) entre procesos.
- **Fallo:** la correcta operación de un sistema distribuido se ve amenazada donde aparezca un fallo en cualquier computador sobre el que se ejecuta (incluyendo fallos de software) o en la red que los conecta.
- **Seguridad:** la naturaleza modular de los sistemas distribuidos y su extensibilidad los expone a ataques tanto de agentes externos como internos.

Modelo de Interacción

- **Concepto de Algoritmo**

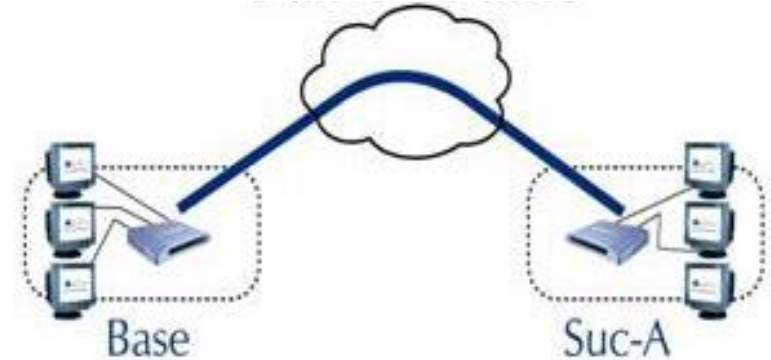


¿Y algoritmo distribuido?

Modelo de Interacción

Factores que afectan a los procesos de un SD

1. **Prestaciones de los canales de comunicaciones:** las características de prestaciones de la comunicación sobre una red de computadores incluyen ***la latencia, el ancho de banda y las fluctuaciones:***
 - a. El retardo entre el envío de un mensaje por un proceso y su recepción por otro se denomina **latencia**. La latencia incluye:
 1. El tiempo que se toma en llegar a su destino el primero de una trama de bits transmitidos a través de una red.
 2. El retardo en acceder a la red, que es grande cuando la red está muy cargada. Por ejemplo, al transmitir sobre Ethernet la estación de emisión esperará a que la red esté libre de tráfico.
 3. El tiempo empleado por los servicios de comunicación en el proceso que envía como en el que recibe en cada sistema operativo.



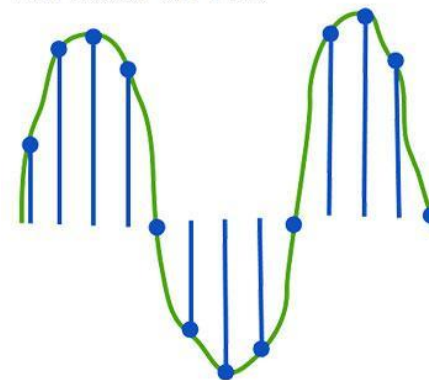
o tanto
ual de

Modelo de Interacción

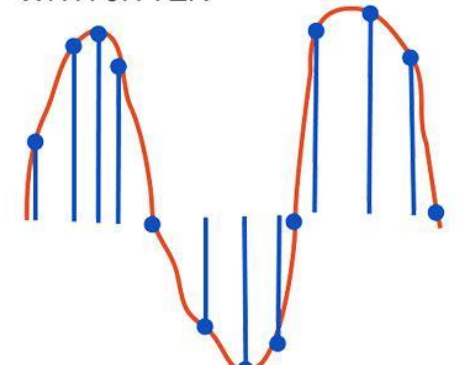
Factores que afectan a los procesos de un SD

- b. **El ancho de banda** de una red de computadores es la cantidad total de información que puede transmitirse en un intervalo de tiempo dado. Cuando una red está siendo utilizada por un número grande de canales de comunicación, habrá que compartir el ancho de banda disponible.
- c. **La fluctuación** (jitter) es la variación en el tiempo invertido en completar la entrega de una serie de mensajes. La fluctuación es importante para los datos multimedia. Éste es el caso que si se reproducen muestras de audio consecutivas a un ritmo variable el sonido presentará graves distorsiones, se requiere un flujo constante.

WITHOUT JITTER



WITH JITTER

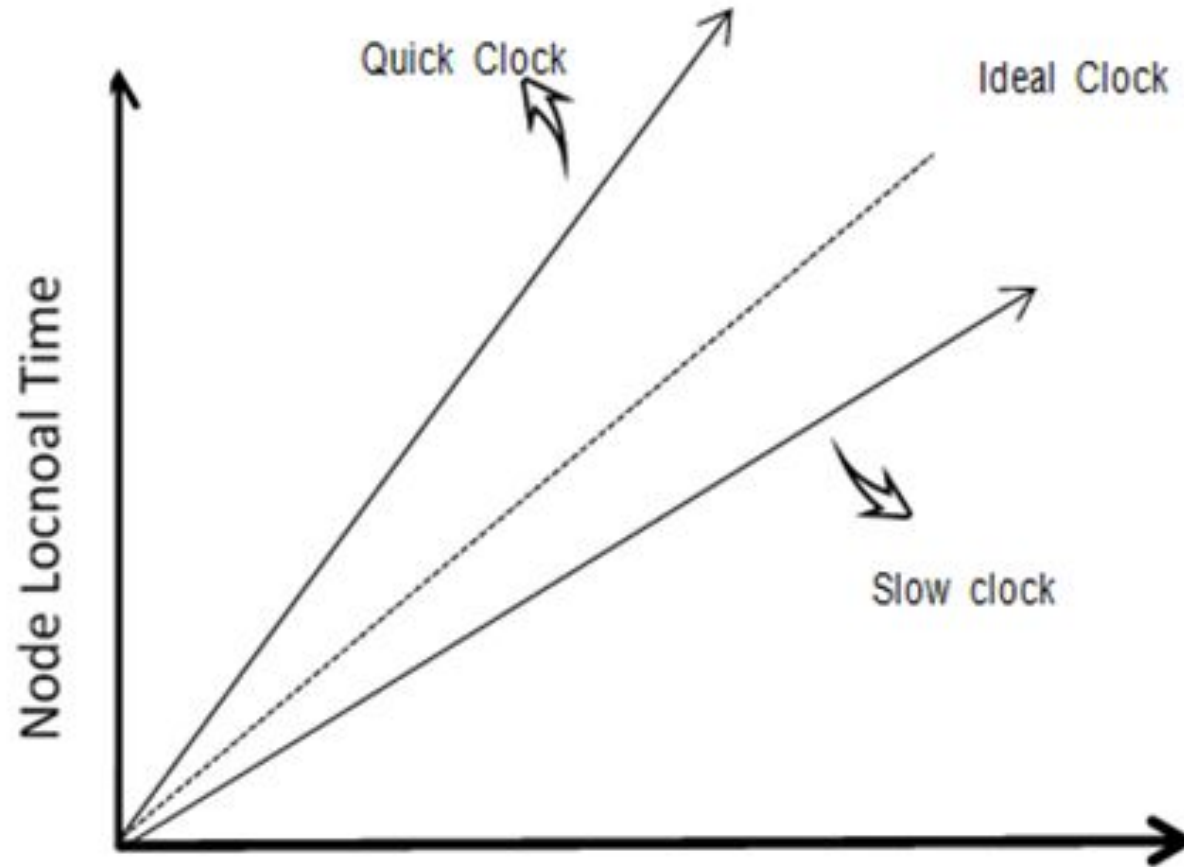


2. **Relojes de computadores y eventos de temporización:** Cada computador de un sistema distribuido tiene su propio reloj interno; los procesos locales obtienen de él, el valor del tiempo actual.
 - Esta es la forma en que dos procesos en ejecución sobre dos computadores diferentes asocian marcas (o sellos) temporales a sus eventos. Sin embargo, incluso si dos procesos leen sus relojes a la vez, sus relojes locales proporcionarán valores de tiempo diferentes.
 - Esto ocurre porque los relojes presentan derivas con respecto al tiempo perfecto y, más importante, sus tasas de deriva difieren de una a otra.
 - **Tasa de deriva del reloj (clock drift rate):** proporción en que el reloj de un computador difiere del reloj de referencia perfecto. Incluso, si los relojes de todos los computadores de un sistema distribuido se ponen en hora a la vez, a partir de un tiempo sus relojes variarán en una magnitud significativa a menos que se apliquen correcciones.

Modelo de Interacción

Factores que afectan a los procesos de un SD

- Clock Drift Rate



Modelo de Interacción

Variantes



3. **Dos variantes del modelo de interacción.** En un sistema distribuido es difícil establecer cotas sobre el tiempo que debe tomar la ejecución de un proceso, el reparto de un mensaje o la deriva del reloj. Tenemos dos modelos simples que parten de posiciones extremas y opuestas: el primero tiene en cuenta una fuerte restricción sobre el tiempo; en el segundo no se hace ninguna presuposición.
- a. **Sistemas distribuidos síncronos:** Hadzilacos y Toueg [1994] definen un sistema distribuido síncrono como aquel en el que se establecen los siguientes límites:
1. El tiempo de ejecución de cada etapa de un proceso tiene ciertos límites inferior y superior conocidos.
 2. Cada mensaje transmitido sobre un canal se recibe en un tiempo limitado conocido.
 3. Cada proceso tiene un reloj local cuya tasa de deriva sobre el tiempo real tiene un límite conocido.

Modelo de Interacción

Variantes

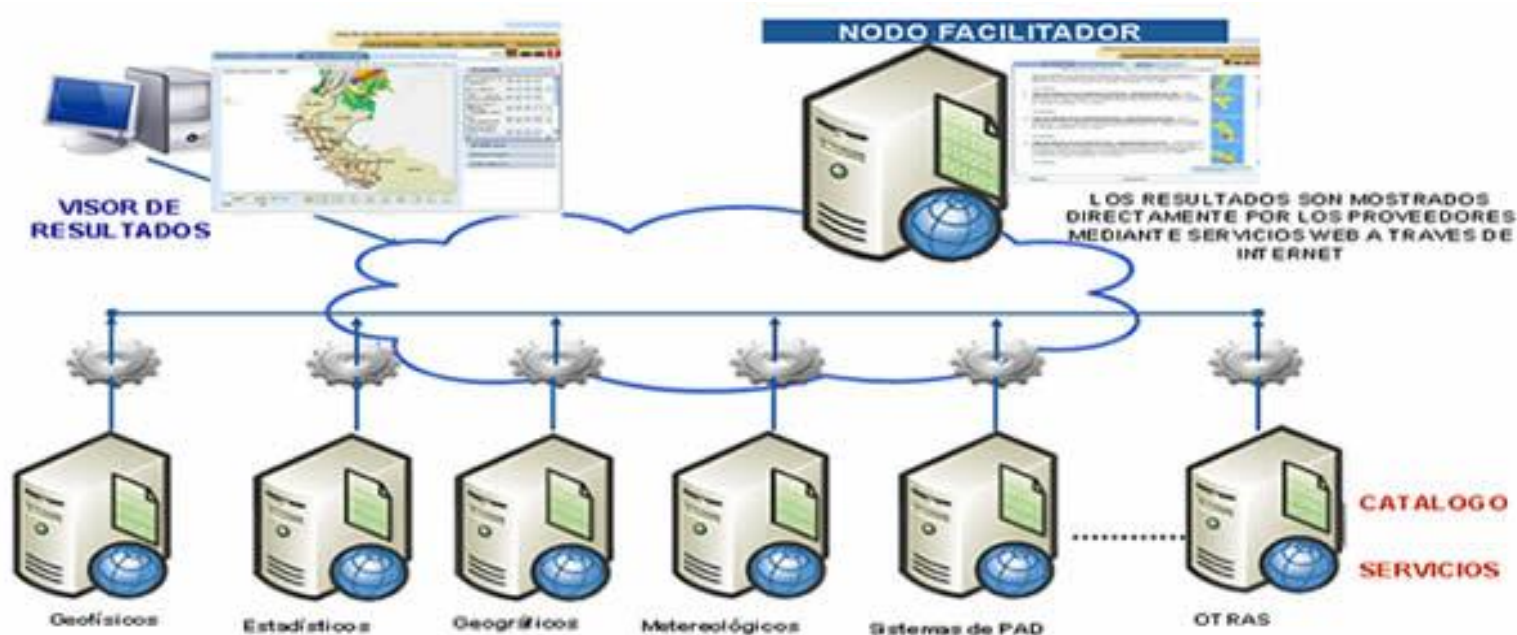


- b. **Sistemas distribuidos asíncronos:** muchos sistemas distribuidos, ejemplo Internet, son de gran utilidad aun sin ser sistemas síncronos. En consecuencia necesitamos un modelo alternativo: un sistema distribuido asíncrono es aquel en que no existen limitaciones sobre:
1. La velocidad de procesamiento (ejemplo, un paso de un proceso puede requerir tan sólo un pico segundo y otro un siglo; lo único que podemos decir es que cada paso se tomará un tiempo arbitrariamente largo).
 2. Los retardos de transmisión de mensaje (por ejemplo, al repartir un mensaje de un proceso A a otro B puede tardar un tiempo cero o bien varios años. En otras palabras, un mensaje puede recibirse tras un tiempo arbitrariamente largo).
 3. Las tasas de deriva de reloj (de nuevo, la tasa de deriva de reloj es arbitraria).

Modelo de Interacción

Variantes

- El modelo asíncrono no presupone nada sobre los intervalos de tiempo involucrados en cualquier ejecución. Éste es exactamente el modelo de Internet, en él no hay límite intrínseco en la carga del servidor o la red (en consecuencia no se sabe cuánto tomará, por ejemplo, transferir un archivo usando ftp). A veces un mensaje de correo puede tomarse varios días en llegar.



Modelo de Interacción

Ordenamiento de Eventos



- En muchos casos, nos interesa saber si un evento (enviar o recibir un mensaje) en un proceso ocurrió antes, después o concurrentemente con otro evento en algún otro proceso. La ejecución de un sistema puede describirse en términos de los eventos y su ordenación aun careciendo de relojes precisos.

Modelo de Interacción

Ordenamiento de Eventos

- Por ejemplo, considere el siguiente conjunto de intercambios entre un pequeño grupo de usuarios X, Y, Z y A de una lista de correo:
 - El usuario X envía un mensaje con el tema Reunión.
 - Los usuarios Y y Z responden con un mensaje con el tema Re: Reunión.

a. Inbox de A

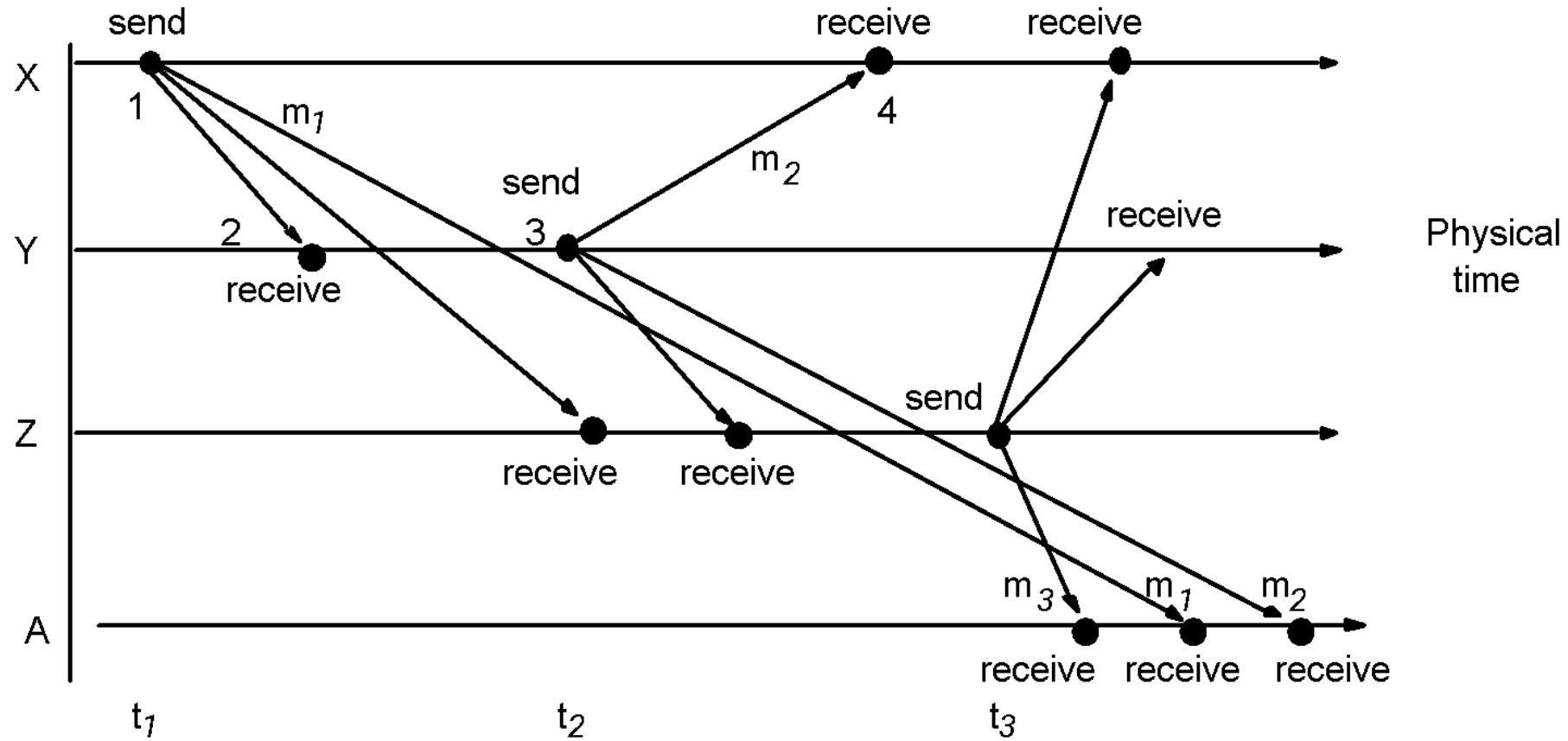
Item	De	Asunto
1	X	Reunión
2	Y	Re: Reunión
3	Z	Re: Reunión

b. Inbox de A

Item	De	Asunto
1	Z	Re: Reunión
2	X	Reunión
3	Y	Re: Reunión

Modelo de Interacción

Ordenamiento de Eventos



Modelo de Interacción

Ordenamiento de Eventos

- Si los relojes de los computadores de X, Y y Z pudieran sincronizarse, podría utilizarse el tiempo asociado localmente a cada mensaje enviado. Por ejemplo, los mensajes m1, m2 y m3 llevarían la tiempos t_1 , t_2 y t_3 donde $t_1 < t_2 < t_3$. Los mensajes recibidos se mostrarían a los usuarios según este ordenamiento temporal.
- Si los relojes están sincronizados aproximadamente la temporización ocurrirá en el orden correcto.
- Como los relojes de un sistema distribuido no pueden sincronizarse de manera perfecta, Lamport [1987] propuso un modelo de tiempo lógico que pudiera utilizarse para ordenar los eventos de procesos que se ejecutan en computadores diferentes de un sistema distribuido.
- El tiempo lógico permite inferir el orden en que se presentan los mensajes sin recurrir a relojes.

Modelo de Interacción

Ordenamiento de Eventos - Lamport

- Es obvio que los mensajes se reciben después de su envío, así podemos admitir un ordenamiento lógico para los pares de sucesos que se muestran en la Figura; por ejemplo, considerando sólo los eventos relativos a X e Y:
- También sabemos que las respuestas se reciben después de que se reciben los mensajes, de modo que tenemos el siguiente ordenamiento lógico para Y:
 - X envía m1, antes de que Y reciba m1 ;
 - Y envía m2 antes de que X reciba m2.
 - Y recibe m1 antes de enviar m2.
- El tiempo lógico lleva esta idea más lejos asignando un número a cada evento, que se corresponde con su ordenamiento lógico, de modo que los últimos eventos tendrán números mayores que los primeros. Como ejemplo, en la Figura se consignan los números 1 a 4 en los eventos en X e Y.

Modelo de Fallo

Fallos por omisión



- “*Define las formas en que puede ocurrir un Fallo.*”
- **Fallos por omisión.** Las faltas clasificadas como fallos por omisión se refieren a casos en los que los procesos o los canales de comunicación no consiguen realizar acciones que se suponen que pueden hacer .
 - **de procesos:** El principal fallo por omisión de un proceso es el fracaso o ruptura accidentada del procesamiento (crash). Cuando decimos que un proceso se rompe queremos decir que ha parado y no ejecutará ningún paso de programa más.

Modelo de Fallo

Fallos por omisión



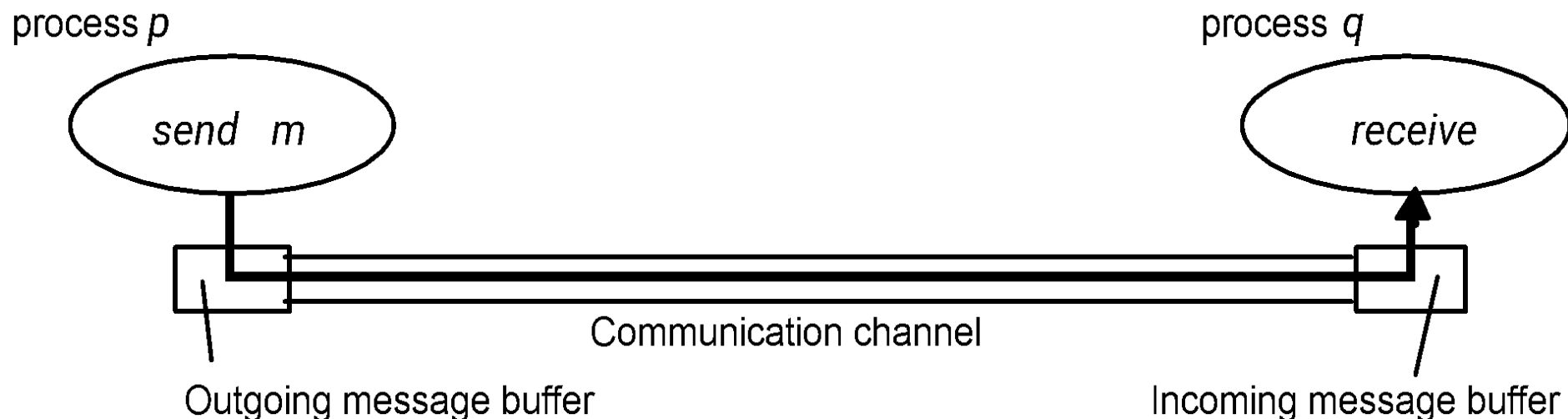
- Otros procesos pueden ser capaces de detectar tales fracasos por el hecho de que los procesos rotos fallan repetidamente en responder a los mensajes de invocación.
- Desgraciadamente, este método de detección de rupturas descansa en el uso de **timeouts**; es decir un método en el cual un proceso otorga un período fijo de tiempo para que algo ocurra. En un sistema asíncrono un timeout puede indicar tan sólo que un proceso no responde; puede que se haya roto o sea lento, o que los mensajes no hayan llegado.
- La rotura de un proceso se denomina fallo-parada (***fail-stop***) si los otros procesos pueden detectar con certeza que el proceso ha fracasado.

Modelo de Fallo

Fallos por omisión



- **de comunicaciones:** El canal de comunicación produce un fallo de omisión si no transporta un mensaje desde el búfer de mensajes salientes de p al búfer de mensajes entrantes de q . A esto se denomina «perder mensajes» y su causa suele ser la falta de espacio en el búfer de recepción de alguna pasarela de entre medias, o por un error de red, detectable por una suma de chequeo de los datos del mensaje.

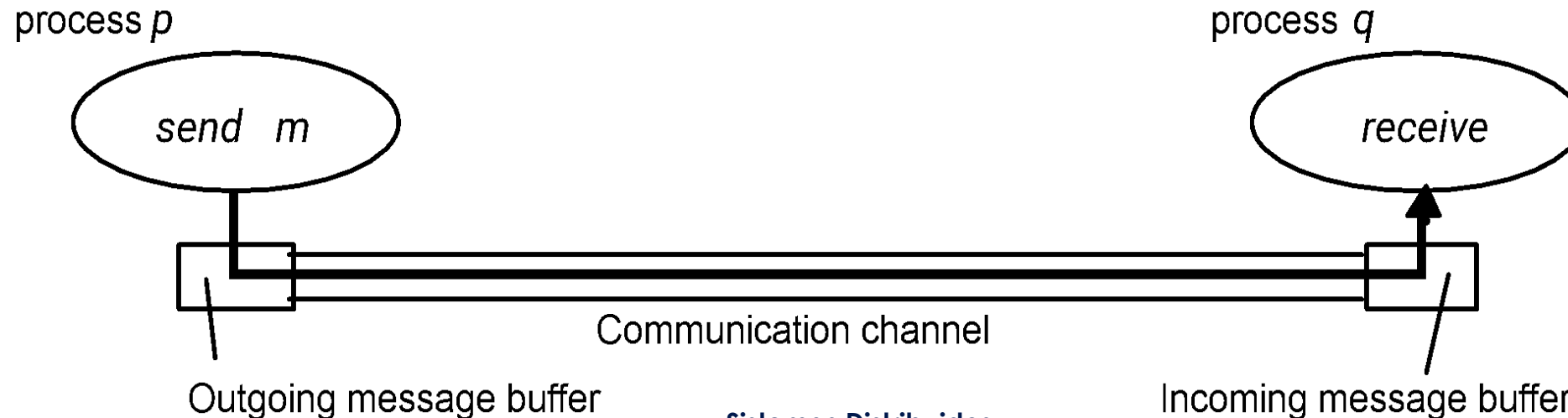


Modelo de Fallo

Fallos por omisión



- Hadzilacos y Toueg [1994] se refieren:
 - **Fallos por omisión de envío:** pérdida de mensajes entre el proceso emisor y el búfer de mensajes de salida.
 - **Fallos por omisión de recepción:** la pérdida de mensajes entre el búfer de mensajes de entrada y el proceso receptor como.
 - **Fallos por omisión del canal:** la pérdida de mensajes entre los búferes.



Modelo de Fallo

Fallos arbitrarios



- El término arbitrario, o fallo bizantino, se emplea para describir la peor semántica de fallo posible, en la que puede ocurrir cualquier tipo de error.
- Un fallo arbitrario en un proceso es aquel en el que se omiten pasos deseables para el procesamiento o se realizan pasos no intencionados de procesamiento. En consecuencia los fallos arbitrarios en los procesos no pueden detectarse observando si el proceso responde a las invocaciones, dado que podría omitir arbitrariamente la respuesta.
- Ejemplos de fallos bizantinos en canales:
 - Contenidos corruptos de mensajes
 - Mensajes no existentes
 - Mensajes duplicados.

Modelo de Fallo

Resumen de Fallos por Omisión y Arbitrarios

Omission and arbitrary failures

<i>Class of failure</i>	<i>Affects</i>	<i>Description</i>
Fail-stop	Process	Process halts and remains halted. Other processes may detect this state.
Crash	Process	Process halts and remains halted. Other processes may not be able to detect this state.
Omission	Channel	A message inserted in an outgoing message buffer never arrives at the other end's incoming message buffer.
Send-omission	Process	A process completes a <i>send</i> operation but the message is not put in its outgoing message buffer.
Receive-omission	Process	A message is put in a process's incoming message buffer, but that process does not receive it.
Arbitrary (Byzantine)	Process or channel	Process/channel exhibits arbitrary behaviour: it may send/transmit arbitrary messages at arbitrary times or commit omissions; a process may stop or take an incorrect step.

Modelo de Fallo

Fallos de temporización y Fiabilidad

- **Fallos de temporización.** Los fallos de temporización se aplican en los sistemas distribuidos síncronos donde se establecen límites en el tiempo de ejecución de un proceso, en el tiempo de reparto de un mensaje y en la tasa de deriva de reloj.

<i>Clase de fallo</i>	<i>Afecta a</i>	<i>Descripción</i>
Reloj	Proceso	El reloj local del proceso excede el límite de su tasa de deriva sobre el tiempo real.
Prestaciones	Proceso	El proceso excede el límite sobre el intervalo entre dos pasos.
Prestaciones	Canal	La transmisión de un mensaje toma más tiempo que el límite permitido.

Figura 2.12. Fallo de temporización.

Modelo de Fallo

Fallos de temporización y Fiabilidad

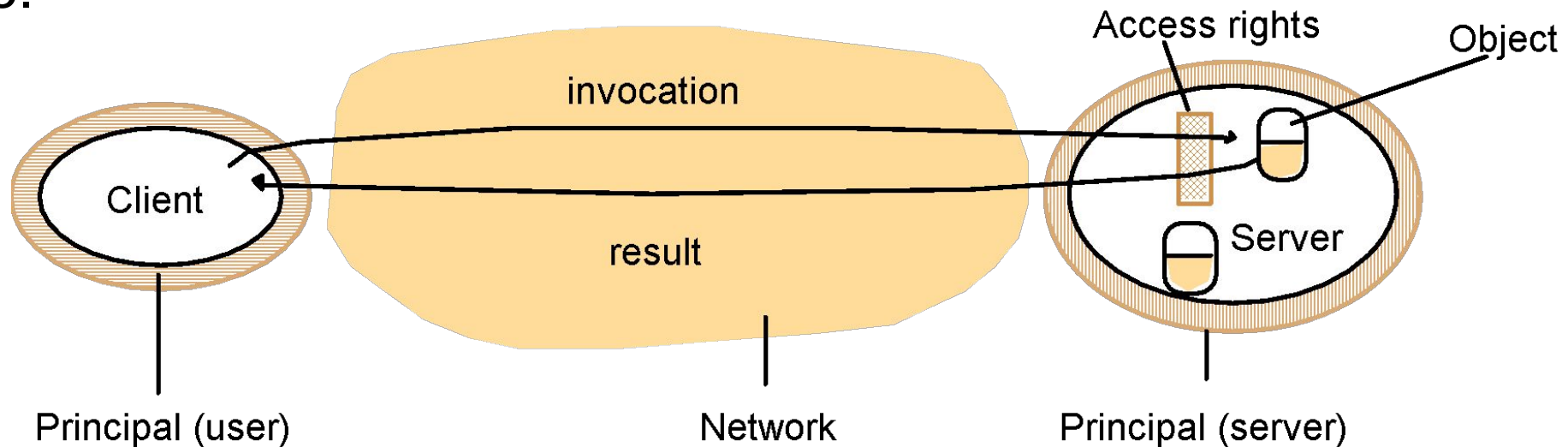


- **Fiabilidad y comunicación uno a uno.** El término comunicación fiable se define en términos de validez e integridad, definidos como sigue:
 - Validez: Cualquier mensaje en el búfer de mensajes salientes eventualmente llegará al búfer de mensajes entrantes;
 - Integridad: El mensaje recibido es idéntico al enviado, y no se entregan mensajes por duplicado.

Modelo de Seguridad

Protección de Objetos

- La seguridad de un sistema distribuido puede lograrse asegurando los procesos y los canales empleados para sus interacciones y protegiendo los objetos que encapsulan contra el acceso no autorizado.
- Protección de objetos.** Los usuarios pueden lanzar programas cliente que envían invocaciones al servidor para realizar operaciones sobre los objetos. El servidor realiza la operación indicada en cada invocación y envía el resultado al cliente.



Modelo de Seguridad Ataques

- **Asegurar procesos y sus interacciones.** Los procesos interaccionan enviando mensajes. Los mensajes están expuestos a un ataque dado que la red y el servicio de comunicación que usan es abierto, de modo que cualquier par de procesos puedan interactuar. Los servidores y procesos parejos exponen sus interfaces, permitiendo ser destino de los mensajes de otros procesos.

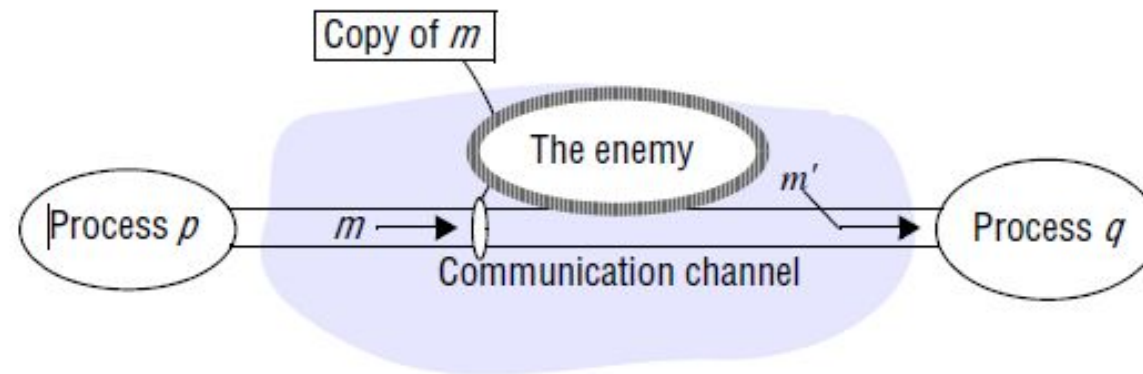


Modelo de Seguridad

El enemigo

- El enemigo. Es un adversario capaz de enviar cualquier mensaje a cualquier proceso y también de leer o copiar cualquier mensaje entre un par de procesos.
- Tales ataques pueden cometerse simplemente utilizando un computador conectado a una red para lanzar un programa que lea los mensajes de la red.

The enemy



Modelo de Seguridad

El enemigo



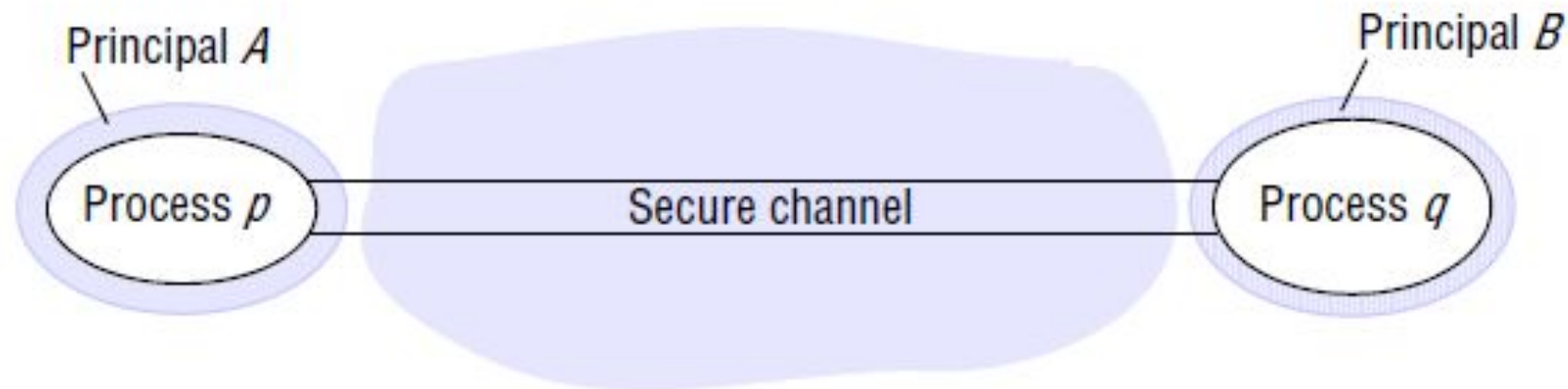
- Amenazas del enemigo
 - **A procesos:** Cualquier proceso diseñado para admitir peticiones puede recibir un mensaje de cualquier otro proceso del sistema distribuido, y bien pudiera no ser capaz de determinar la identidad del emisor.
 - **A Servidores:** dado que un servidor puede recibir invocaciones de muchos clientes diferentes, no necesariamente puede determinar la identidad del principal que se halla tras cualquier invocación particular.
 - **A Clientes:** cuando un cliente recibe el resultado de una invocación de un servidor, no necesariamente puede decir si la fuente del mensaje resultado es del servidor que se pretendía o de un enemigo (spoofing).
 - **A los canales de comunicación:** Un enemigo puede copiar, alterar o insertar mensajes según viajan a través de la red y sus pasarelas.
- Todas estas amenazas pueden vencerse empleando **canales seguros**, y que se basan en la criptografía y la autenticación.

Modelo de Seguridad

Canales Seguros

- La encriptación y la autenticación se emplean para construir canales seguros en forma de capa de servicio sobre los servicios de comunicación existentes. Un canal seguro es un canal de comunicación que conecta un par de procesos, cada uno de los cuales actúa en representación de un principal.

Secure channels



Modelo de Seguridad

Canales Seguros



- Un canal seguro presenta las siguientes propiedades:
 - Cada proceso conoce bien la identidad del principal en cuya representación se ejecuta otro proceso. Si un cliente y un servidor se comunican vía un canal seguro, el servidor conoce la identidad del principal tras las invocaciones y puede comprobar sus derechos de acceso antes de realizar una operación. Esto permite que el servidor proteja sus objetos correctamente y permite al cliente estar seguro de estar recibiendo resultados de un servidor que actúa de buena fe.
 - Un canal seguro asegura la privacidad y la integridad (protección contra la manipulación) de los datos transmitidos por él.
 - Cada mensaje incluye un sello de carácter temporal, de tipo físico o lógico, para prevenir el reenvío o la reordenación de los mensajes.

Modelo de Seguridad

Otras posibles amenazas

- **Denegación de servicio:** ésta es una forma de ataque en la que el enemigo interfiere con las actividades de los usuarios autorizados mediante un número excesivo de invocaciones sin sentido sobre servicios o la red, la que resulta en una sobrecarga de los recursos físicos (ancho de banda, capacidad de procesamiento del servidor).
- **Código Móvil:** un código descargado del servidor y ejecutado en el cliente podría actuar como TrojanHorse



Fin

- Propuesta de Ejercicios

